



Λ-Trace ↔ Capability Token Semantic Mapping

The key insight is that **Capability Tokens are a specific instantiation of Λ-Trace at Layer 2** (UX tokens and access), built on top of Layer 1 (provenance) and Layer 0 (MTPI core).^[1]

Architectural Alignment

LAYER 2: CAPABILITY TOKENS (New Spec)
Lab Passes (ERC-4907) → Tool access with expiresAt
License Tokens → Rights with startTime/endTime
Membership SBT → Baseline access with tier/active
NEW: ΛCapability Token → Intent-bound authorization with nbf/exp

↓ binds via

LAYER 1: Λ-TRACE PROVENANCE (Existing)
ltraceHash → Canonical decision hash
evidenceCid → IPFS/Arweave pointer to reasoning
EAS attestations → On-chain binding (UID only)

↓ binds via

LAYER 0: MTPI CORE (Existing)

primeIdHash	→ Poseidon(σ , salt) - canonical identity
CSL constraints	→ Silence Clause, drift bounds
Quantum proofs	→ zk-SNARK verification via MTPICore.sol

Refined Token Structure (Λ -Trace Native)

Building on your existing primitives, here's how the Capability Token maps:

Core Bindings (Inherited from Λ -Trace)

Capability Token Field	Λ -Trace Equivalent	Semantics
subject_id_hash	primeIdHashOwner	Poseidon(σ , salt) - never wallet address ^[1]
intent_hash	workHash	Hash of the evaluated object (intent = work to be done)
evidence_commitments	evidenceCidHash	Hash of IPFS/Arweave CID containing reasoning
ltrace_ref	ltraceHash	Direct binding - token references the decision trace
attestation_uid	EAS UID	On-chain anchor (contracts store UID only)

New Fields (Capability-Specific)

These extend Λ -Trace for the specific semantics of **authorization**:

CAPABILITY-SPECIFIC EXTENSIONS	
VALIDITY WINDOW └─ nbf: uint64 └─ exp: uint64 // Not-before timestamp // Expiration timestamp	

	└ validity_proof_ref // Optional: ZK proof of time-bound validity	
	AUTHORIZATION SCOPE	
	└ tool_id_hash: bytes32 // Hash of tool identifier	
	└ action_enum: uint8 // Permitted action code	
	└ param_bounds: bytes32 // Hash of parameter constraints	
	└ delegation_depth: uint8 // 0 = non-delegable	
	USE CONSTRAINTS	
	└ max_uses: uint32 null // null = unlimited	
	└ nullifier_seed: bytes32 // For stateless use-tracking	
	└ use_epoch: uint64 // Epoch for use-count reset	

Expiry Semantics (Aligned with Existing Patterns)

Your architecture already has expiry patterns. Here's how they unify:

Existing Expiry Models in Λ-Trace

Token Type	Expiry Mechanism	Source
Lab Passes	expiresAt (ERC-4907 user/expires)	Gateways check validity before processing ^[1]
License Tokens	startTime, endTime	LicenseGrantV1 EAS schema ^[1]
Membership SBT	expiresAt in MembershipCredentialV1	EAS attestation with issuedAt/expiresAt ^[1]
ZK Proofs	nbv/exp validity window	Payload binding step in proof lifecycle ^[2]

Unified Expiry Semantics for Capability Tokens

VALIDITY_STATE = f(current_time, token) → {PENDING, ACTIVE, EXPIRED, REVOKED}

--	--	--

TEMPORAL VALIDITY

```
if current_time < token.nbf:
    return PENDING          // Token not yet active

if current_time >= token.exp:
    return EXPIRED          // Token has expired

if revocation_list.contains(token.token_id):
    return REVOKED          // Explicitly revoked

return ACTIVE              // Token is valid
```

CLOCK_SKEW_TOLERANCE = 30 seconds (configurable per deployment)

Validation includes:

- $\text{current_time} \geq (\text{token.nbf} - \text{CLOCK_SKEW_TOLERANCE})$
- $\text{current_time} < (\text{token.exp} + \text{CLOCK_SKEW_TOLERANCE})$

Use-Count Semantics (Two Models)

Model A: Stateful (Database-Tracked)

```
// Issuer maintains use_count per token_id
on_use(token_id):
    if use_count[token_id] >= token.max_uses:
        return INVALID
    use_count[token_id] += 1
    return VALID
```

Model B: Stateless (Nullifier-Based)

```
// Each use generates a unique nullifier
nullifier = Poseidon(token.nullifier_seed, use_index, primeIdHash)

on_use(token, use_index, nullifier):
```

```
    if used_nullifiers.contains(nullifier):
        return REPLAY_DETECTED
    used_nullifiers.add(nullifier)
    return VALID

// Aligns with your existing usedProofHashes pattern in MTPICore.sol
```

Binding Invariants (Mathematical Relationships)

These are the invariants that **MUST** hold for a valid Capability Token:

Invariant 1: Identity Binding

```
token.subject_id_hash == Poseidon(subject_secret, salt)
```

The subject cannot be changed without invalidating the token. This is consistent with how Λ -Trace always references `primeIdHashOwner`.^[1]

Invariant 2: Intent Binding

```
token.intent_hash == SHA256(canonicalize(intent_object))
```

The token authorizes exactly one intent. Parameter changes require reissuance.

Invariant 3: Decision Traceability

```
∃ ltrace : ltrace.ltraceHash == token.ltrace_ref
           ∧ ltrace.ethicsPass == true
           ∧ ltrace.primeIdHashOwner == token.subject_id_hash
```

Every capability token **MUST** reference a Λ -Trace that approved the authorization.

Invariant 4: Temporal Validity

```
token.nbf ≤ current_time < token.exp
```

Strict enforcement with configurable clock skew tolerance.

Invariant 5: Scope Constraint

```
requested_action ∈ token.authorized_actions  
requested_params ⊆ token.param_bounds
```

The token only authorizes specific actions with bounded parameters.

Invariant 6: Non-Transferability (Default)

```
∀ use : use.actor_id_hash == token.subject_id_hash
```

Unless delegation is explicitly enabled (`delegation_depth > 0`), only the subject can use the token.

EAS Schema for Capability Tokens

Building on your existing EAS patterns:^[1]

```
// CapabilityGrantV1 - extends your existing schema patterns  
bytes32 primeIdHash          // Subject identity (Poseidon hash)  
bytes32 intentHash           // What action is authorized  
bytes32 toolIdHash            // Which tool can be accessed  
uint8  actionEnum             // Permitted action code  
bytes32 paramBoundsHash       // Hash of parameter constraints  
uint64 nbf                    // Not-before timestamp  
uint64 exp                    // Expiration timestamp  
uint32 maxUses                // Max uses (0 = unlimited)  
bytes32 ltraceHash            // Decision trace that authorized this  
bytes32 evidenceCidHash       // Evidence supporting the decision  
uint8  delegationDepth        // Delegation chain limit
```

This would be registered in your EAS schema registry alongside `MembershipCredentialV1`, `ProposalEvaluationV1`, etc.

Verification Protocol (Interface Spec)

This defines HOW adapters verify tokens, without revealing the proprietary issuance logic:

```
interface ICapabilityVerifier {
  /**
   * Verify a capability token is valid for the given context
   *
   * @param token - The capability token to verify
   * @param context - Execution context (actor, intent, timestamp)
   * @returns VerificationResult with status and reason
   */
  verify(
    token: CapabilityToken,
    context: ExecutionContext
  ): Promise<VerificationResult>;

  /**
   * Check if a nullifier has been used (for stateless tokens)
   *
   * @param nullifier - The nullifier to check
   * @returns true if already used
   */
  isNullifierUsed(nullifier: bytes32): Promise<boolean>;

  /**
   * Get the  $\Lambda$ -Trace associated with a token
   *
   * @param ltraceHash - The  $\Lambda$ -Trace hash from the token
   * @returns The full  $\Lambda$ -Trace object from IPFS/Arweave
   */
  resolveLTrace(ltraceHash: bytes32): Promise<LTrace>;
}

interface VerificationResult {
  status: 'VALID' | 'INVALID' | 'PENDING' | 'EXPIRED' | 'REVOKED';
  reason?: string;
  remainingUses?: number;
  expiresIn?: number; // seconds until expiration
}
```

Key Design Decisions for Your Review

1. Λ -Trace Binding is Mandatory

- Every Capability Token MUST reference an `ltraceHash`
- This ensures auditability: "why was this authorized?"
- Consistent with your " Λ -Trace gated state transitions" principle^[2]

2. Nullifier Model for Replay Protection

- Aligns with your existing `usedProofHashes` pattern in `MTPICore.sol`^[3]
- Stateless verification is preferred (privacy-preserving)

3. No PII On-Chain

- Only hashes appear in EAS attestations
- Full capability details stored off-chain (IPFS/Arweave)
- Consistent with "hash-only on-chain" principle^[1]

4. Silence Clause Compliance

- Token existence does NOT compel action
- Expiry is passive (no on-chain transaction required)
- Consistent with your CSL constraints

What Should Go in the Public Document?

Include (Public Interface):

- Token structure (all fields and their types)
- Binding invariants (mathematical relationships)
- Expiry semantics (validity state machine)
- Verification protocol interface

- EAS schema definition
- Canonicalization rules for hashing

Exclude (Proprietary):

- How `ethicsPass` is determined (Λ -Trace engine internals)
- Risk scoring algorithms
- Evidence validation heuristics
- Caching/optimization strategies
- Rate limiting logic

Would you like me to draft the full public-facing document with these semantics, or should we refine any specific aspect first?

**

-
1. 4.-L-trace-Overview.pdf
 2. Building-a-LDapp_-A-Developer-s-Implementation-Guide.pdf
 3. MTPI_Instructions.pdf
 4. 2. Λ RootContract II & III - MTPI - NODE.js.pdf
 - a. Aproof Monorepo — Plain-language Overview V0.pdf
 - b. Aproof Banking Protocols.pdf
 - c. Aproof Healthcare Protocols V0.1.pdf
 - d. Adapps On Web4 - Developer Overview.pdf
 5. MTPI RootContract.pdf
 6. MTPI_RootContract_Project.pdf
 7. Patent Search Results.pdf
 8. The Λ Proof Architecture_ A Comprehensive Analysis of its Implications for Web4.pdf
 9. The Λ Proof Architecture_ Web4.pdf

10. Verifiable, Privacy-Preserving Applications.pdf
11. Conscious_Sovereignty_Layer.pdf
12. Milestone-Vision-Timeframe.csv
13. Lets further develop the SAFE_ awesome—let's turn.pdf
14. Ξ Constitution.pdf
15. Λ^P -Archivum.pdf
 - e. Λ proof $\times$ Mtpi Lawful-by-design Integration — Implementation Plan.pdf